CASE STUDY 5

Unit V: Graphs and Trees

GPS Navigation Using a Weighted Graph and Huffman Coding

Aim

To use a weighted graph to find a shortest route and understand frequency-based Huffman encoding.

Objectives

- Represent places and roads as a weighted graph.
- Find a shortest route using Dijkstra's algorithm.
- Understand the basic principle of Huffman coding.

Theory

A weighted graph represents locations as vertices and roads as edges with weights such as distance. Dijkstra's algorithm finds a shortest path when all edge weights are non-negative. Huffman coding is a lossless coding method that assigns shorter codes to more frequent symbols. The syllabus mentions GPS/navigation and ZIP/GZIP compression as applications of graphs and trees.

Algorithm

1. GPS: initialize distances, repeatedly choose the unvisited vertex with the smallest distance, and update distances to its neighbours.

2. Huffman: count symbol frequencies, repeatedly combine the two least-frequent nodes to build a binary tree, then assign codes to its branches.

Python Program

```
# Program 1: Shortest route using Dijkstra's algorithm
graph = {
    "A": {"B": 4, "C": 10},
    "B": {"A": 4, "C": 3, "D": 8},
    "C": {"A": 10, "B": 3, "D": 2},
    "D": {"B": 8, "C": 2}
}

def dijkstra(graph, start):
    distances = {node: float("inf") for node in graph}
    previous = {}
    unvisited = list(graph)
    distances[start] = 0

    while unvisited:
        current = min(unvisited, key=lambda node: distances[node])
        unvisited.remove(current)

        if distances[current] == float("inf"):
            break

        for neighbour, weight in graph[current].items():
            new_distance = distances[current] + weight
            if new_distance < distances[neighbour]:
                distances[neighbour] = new_distance
                previous[neighbour] = current

    return distances, previous

distances, previous = dijkstra(graph, "A")
destination = "D"
path = []
current = destination
while current in previous:
    path.append(current)
    current = previous[current]
path.append(current)
path.reverse()
```

```
print("Shortest distance A to D:", distances["D"])
print("Route:", " -> ".join(path))


# Program 2: Show symbol frequencies for Huffman coding
from collections import Counter

message = "BANANA"
frequencies = Counter(message)
print("Symbol frequencies:")
for symbol, count in sorted(frequencies.items()):
    print(symbol, ":", count)
```

Sample Output

```
Shortest distance A to D: 9\nRoute: A -> B -> C -> D\nSymbol frequencies:\nA : 3\nB : 1\nN : 2
```

Conclusion

Weighted graphs help calculate routes based on distance or cost. Huffman trees use symbol frequencies to create efficient prefix codes. Real ZIP/GZIP formats use DEFLATE, which combines LZ77 and Huffman coding.